

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Carrera: Ingeniería en Ciencias y Sistemas

Curso: Sistemas Operativos 2



Práctica 1

201830006 Dany Noé de León Santizo

Quetzaltenango, 18 de agosto del 2026

Repo https://github.com/DNLS89/Practica2_SOPES2.git

Ejercicio 1

Código:

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/wait.h>
5
6/*
7 * Ejercicio 1: Comunicación mediante pipes.
8 *
9 * El programa utiliza dos procesos:
10 * - Padre: representa al cliente
11 * - Hijo: representa al sistema que procesa la información
12 *
13 * Se utilizan dos pipes para permitir comunicación en ambos sentidos:
14 * - pipe_padre_hijo: padre -> hijo
15 * - pipe_hijo_padre: hijo -> padre
16 *
17 * También se utiliza un indicador para que el hijo pueda identificar
18 * qué tipo de información está recibiendo
19 */
20
21/*
22 * Invierte una cadena carácter por carácter
23 */
24void invertir_cadena(char *cadena, char *invertida)
25{
26    int longitud = 0;
27
28    // Calculamos la longitud de la cadena
29    while (cadena[longitud] != '\0') {
30        longitud++;
31    }
32
33    // Copiamos los caracteres comenzando desde el último
34    for (int i = 0; i < longitud; i++) {
35        invertida[i] = cadena[longitud - 1 - i];
36    }
37
38    invertida[longitud] = '\0';
39}
40
41int main()
42{
43    int pipe_padre_hijo[2];
44    int pipe_hijo_padre[2];
45
46    // Creamos las dos tuberías
47    if (pipe(pipe_padre_hijo) == -1 || pipe(pipe_hijo_padre) == -1) {
48        perror("Error al crear las tuberías");
49        return 1;
50    }
51
52    // Creamos el proceso hijo
53    pid_t pid = fork();
54
55    if (pid == -1) {
56        perror("Error al crear el proceso hijo");
57        return 1;
58    }
59
60    // =====
61    // PROCESO PADRE
62    // =====
63    if (pid > 0) {
64
65        // El padre solamente escribe en el primer pipe y solo lee del segundo
66        close(pipe_padre_hijo[0]);
67        close(pipe_hijo_padre[1]);
68
69        int opcion;
70
71        printf("=== SISTEMA DE PAGO ===\n");
72        printf("1. Verificar estado del canal\n");
73        printf("2. Omitir verificación\n");
74        printf("Seleccione una opción: ");
75        scanf("%d", &opcion);
76
77        /*
78         * Si el usuario quiere verificar el canal, enviamos el indicador 1
79         */
80        if (opcion == 1) {
81
82            int indicador = 1;
83
84            write(pipe_padre_hijo[1], &indicador, sizeof(indicador));
85
86            char mensaje[100];
87            char respuesta[100];
```

```

88
89     printf("\nEscriba una palabra para verificar el canal: ");
90     scanf("%s", mensaje);
91
92     // Enviamos el mensaje al hijo
93     write(pipe_padre_hijo[1], mensaje, sizeof(mensaje));
94
95     // Esperamos la respuesta del hijo
96     read(pipe_hijo_padre[0], respuesta, sizeof(respuesta));
97
98     printf("Confirmacion del canal: %s\n", respuesta);
99 }
100 else if (opcion != 2) {
101
102     printf("Opcion no valida.\n");
103
104     close(pipe_padre_hijo[1]);
105     close(pipe_hijo_padre[0]);
106
107     wait(NULL);
108     return 0;
109 }
110
111 /*
112  * Ahora comienza la fase de pago
113  * El indicador 2 informa al hijo que el siguiente dato
114  * corresponde al número de tarjeta
115  */
116 int indicador = 2;
117
118 write(pipe_padre_hijo[1], &indicador, sizeof(indicador));
119
120 int tarjeta;
121
122 do {
123     printf("\nIngrese su numero de tarjeta (1000 - 9999): ");
124     scanf("%d", &tarjeta);
125
126     if (tarjeta < 1000 || tarjeta > 9999) {
127         printf("Numero fuera de rango.\n");
128     }
129
130 } while (tarjeta < 1000 || tarjeta > 9999);
131
132 // Enviamos el número de tarjeta al hijo
133 write(pipe_padre_hijo[1], &tarjeta, sizeof(tarjeta));

```

```

134
135 // Recibimos el resultado del hijo
136 char resultado[50];
137
138 read(pipe_hijo_padre[0], resultado, sizeof(resultado));
139
140 printf("\nResultado final: %s\n", resultado);
141
142 // Cerramos los pipes
143 close(pipe_padre_hijo[1]);
144 close(pipe_hijo_padre[0]);
145
146 // Esperamos que termine el hijo
147 wait(NULL);
148 }
149
150 // =====
151 // PROCESO HIJO
152 // =====
153 else {
154
155     // El hijo solamente lee del primer pipe y solo escribe en el segundo
156     close(pipe_padre_hijo[1]);
157     close(pipe_hijo_padre[0]);
158
159     int indicador;
160
161     /*
162     * El hijo permanece leyendo indicadores hasta recibir
163     * la instrucción correspondiente al pago
164     */
165     while (read(pipe_padre_hijo[0], &indicador, sizeof(indicador)) > 0) {
166
167         // -----
168         // Indicador 1: verificar el canal
169         // -----
170         if (indicador == 1) {
171
172             char mensaje[100];
173             char invertida[100];
174
175             // Recibimos el mensaje enviado por el padre
176             read(pipe_padre_hijo[0], mensaje, sizeof(mensaje));
177
178             // Invertimos el mensaje
179             invertir_cadena(mensaje, invertida);
180

```

```

181         // Enviamos la respuesta al padre
182         write(pipe_hijo_padre[1], invertida, sizeof(invertida));
183     }
184
185     // -----
186     // Indicador 2: procesar el pago
187     // -----
188     else if (indicador == 2) {
189         int tarjeta;
190
191         // Recibimos el número de tarjeta.
192         read(pipe_padre_hijo[0], &tarjeta, sizeof(tarjeta));
193
194         char resultado[50];
195
196         // Verificamos si el número es par o impar
197         if (tarjeta % 2 == 0) {
198             sprintf(resultado, "PAGO_APROBADO");
199         } else {
200             sprintf(resultado, "PAGO_RECHAZADO");
201         }
202
203         // Enviamos el resultado al padre
204         write(pipe_hijo_padre[1], resultado, sizeof(resultado));
205
206         break;
207     }
208 }
209
210 // Cerramos los pipes
211 close(pipe_padre_hijo[0]);
212 close(pipe_hijo_padre[1]);
213 }
214
215 return 0;
216 }
217 }
218

```

Ejecución:

Caso 1 - Comunicación con verificación

```

> ./ejercicio1
=== SISTEMA DE PAGO ===
1. Verificar estado del canal
2. Omitir verificacion
Seleccione una opcion: 1

Escriba una palabra para verificar el canal: hola
Confirmacion del canal: aloh

Ingrese su numero de tarjeta (1000 - 9999): 1234

Resultado final: PAGO_APROBADO
/run/me/d/0/U/S/De/U 2026/8/C/S0/L/Practica 2/Practica2 SOPES2 main ?3

```

DESCRIPCIÓN: Se envió la palabra “hola” desde el proceso padre hacia el hijo mediante el pipe, y el hijo la invirtió correctamente a “aloh”. Después se ingresó el número 1234, que fue procesado por el hijo y, al ser un número par, se recibió la respuesta PAGO_APROBADO.

Caso 2 – Sin verificación

```
> ./ejercicio1
=== SISTEMA DE PAGO ===
1. Verificar estado del canal
2. Omitir verificacion
Seleccione una opcion: 2

Ingrese su numero de tarjeta (1000 - 9999): 1235

Resultado final: PAGO RECHAZADO
/run/me/d/0/U/S/De/U 2026/8/C/S0/Lab/Practica 2/Practica2_S0PES2 main ?3
> |
```

DESCRIPCION: En esta prueba se decidió omitir la verificación del canal y pasar directamente al pago. Se ingresó el número 1235, que fue enviado al proceso hijo mediante el pipe. Como el número es impar, el hijo respondió PAGO_RECHAZADO y el padre mostró el resultado.

Preguntas de análisis

- ¿Qué ocurriría si el hijo intentara leer de la tubería antes de que el padre haya escrito algo?

El proceso hijo quedaría bloqueado esperando a que el padre escriba información en la tubería. Cuando el padre finalmente envíe los datos, el hijo podrá continuar con la lectura.

- La segunda fase requiere que el hijo envíe una respuesta de vuelta al padre.

¿Por qué no es suficiente con la misma tubería usada en la verificación?

Porque un pipe normalmente se utiliza en un solo sentido. En este caso necesitamos que la información viaje del padre al hijo y luego regrese del hijo al padre. Por eso se utilizan dos pipes: uno para enviar datos y otro para recibir la respuesta.

- ¿Qué problema concreto surgiría si ambos procesos intentaran leer y escribir sobre la misma tubería?

Podría ocurrir que un proceso terminara leyendo los datos que él mismo escribió o que ambos procesos intentaran leer al mismo tiempo. Esto puede provocar bloqueos, pérdida de información o un comportamiento inesperado. Por eso es más seguro utilizar un pipe para cada dirección de comunicación.

Ejercicio 2

Código

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 /*
7  * Ejercicio 2: Productor-Consumidor utilizando pipes.
8  * El proceso padre funciona como productor y registra los 20 pedidos
9  * Los dos procesos hijos representan las estaciones de empaque
10  * Ambos hijos utilizan el mismo pipe para recibir los pedidos
11  * Cada estación toma el siguiente pedido disponible y mantiene su
12  * propio contador de pedidos y total de unidades procesadas
13  */
14
15 int main()
16 {
17     int pipe_pedidos[2];
18
19     // Creamos la tubería que transportará los pedidos
20     if (pipe(pipe_pedidos) == -1) {
21         perror("Error al crear el pipe");
22         return 1;
23     }
24
25     // Creamos el primer hijo
26     pid_t hijo1 = fork();
27
28     if (hijo1 == -1) {
29         perror("Error al crear el primer hijo");
30         return 1;
31     }
32
33     // Creamos el segundo hijo solamente desde el padre
34     pid_t hijo2 = -1;
35
36     if (hijo1 > 0) {
37         hijo2 = fork();
38
39         if (hijo2 == -1) {
40             perror("Error al crear el segundo hijo");
41             return 1;
42         }
43     }
44 }
45
46 // =====
47 // PROCESO PADRE - PRODUCTOR
48 // =====
49 if (hijo1 > 0 && hijo2 > 0) {
50
51     // El padre escribe en el pipe, por lo que no necesita leer
52     close(pipe_pedidos[0]);
53
54     printf("=== REGISTRO DE PEDIDOS ===\n");
55
56     /*
57      * El encargado ingresa los 20 pedidos antes de comenzar
58      * a enviarlos por la banda transportadora
59      */
60     for (int i = 0; i < 20; i++) {
61
62         int unidades;
63
64         do {
65             printf("Pedido %d - unidades (1-100): ", i + 1);
66             scanf("%d", &unidades);
67
68             if (unidades < 1 || unidades > 100) {
69                 printf("Valor invalido. Debe estar entre 1 y 100.\n");
70             }
71
72         } while (unidades < 1 || unidades > 100);
73
74         /*
75          * Guardamos temporalmente cada pedido en el pipe
76          * El dato se escribe inmediatamente después de ingresarlo
77          */
78         write(pipe_pedidos[1], &unidades, sizeof(unidades));
79     }
80
81     /*
82      * Al terminar los 20 pedidos, cerramos el extremo de escritura
83      * Esto permite que los hijos sepan que ya no quedan pedidos
84      */
85     close(pipe_pedidos[1]);
86
87     // Esperamos a que ambas estaciones terminen
88     waitpid(hijo1, NULL, 0);
89     waitpid(hijo2, NULL, 0);
90 }
```



```
// =====
// PROCESO HIJO - ESTACIÓN 1
// =====
else if (hijo1 == 0) {

    // La estación solamente necesita leer del pipe
    close(pipe_pedidos[1]);

    int pedido;
    int cantidad_pedidos = 0;
    int total_unidades = 0;

    /*
     * La estación toma pedidos mientras haya datos disponibles
     * Cuando read() devuelve 0 significa que el padre cerró
     * el extremo de escritura y ya no quedan pedidos
     */
    while (read(pipe_pedidos[0], &pedido, sizeof(pedido)) > 0) {

        cantidad_pedidos++;
        total_unidades += pedido;
    }

    printf("\n=== ESTACION DE EMPAQUE 1 ===\n");
    printf("Pedidos procesados: %d\n", cantidad_pedidos);
    printf("Total de unidades despachadas: %d\n", total_unidades);

    close(pipe_pedidos[0]);
    exit(0);
}

// =====
// PROCESO HIJO - ESTACIÓN 2
// =====
else if (hijo2 == 0) {

    // La estación solamente necesita leer del pipe
    close(pipe_pedidos[1]);

    int pedido;
    int cantidad_pedidos = 0;
    int total_unidades = 0;
```

```
/*
 * Esta estación compete con la estación 1 por los pedidos
 * Cada read() obtiene el siguiente pedido disponible
 */
while (read(pipe_pedidos[0], &pedido, sizeof(pedido)) > 0) {

    cantidad_pedidos++;
    total_unidades += pedido;
}

printf("\n=== ESTACION DE EMPAQUE 2 ===\n");
printf("Pedidos procesados: %d\n", cantidad_pedidos);
printf("Total de unidades despachadas: %d\n", total_unidades);

close(pipe_pedidos[0]);
exit(0);
}

return 0;
}
```

Ejecución

```
> gcc ejercicio2.c -o ejercicio2
> ./ejercicio2
=== REGISTRO DE PEDIDOS ===
Pedido 1 - unidades (1-100): 10
Pedido 2 - unidades (1-100): 20
Pedido 3 - unidades (1-100): 30
Pedido 4 - unidades (1-100): 40
Pedido 5 - unidades (1-100): 50
Pedido 6 - unidades (1-100): 60
Pedido 7 - unidades (1-100): 70
Pedido 8 - unidades (1-100): 80
Pedido 9 - unidades (1-100): 90
Pedido 10 - unidades (1-100): 100
Pedido 11 - unidades (1-100): 10
Pedido 12 - unidades (1-100): 20
Pedido 13 - unidades (1-100): 30
Pedido 14 - unidades (1-100): 40
Pedido 15 - unidades (1-100): 50
Pedido 16 - unidades (1-100): 60
Pedido 17 - unidades (1-100): 70
Pedido 18 - unidades (1-100): 80
Pedido 19 - unidades (1-100): 90
Pedido 20 - unidades (1-100): 100

=== ESTACION DE EMPAQUE 1 ===
=== ESTACION DE EMPAQUE 2 ===
Pedidos procesados: 11
Total de unidades despachadas: 600
Pedidos procesados: 9
Total de unidades despachadas: 500
```

En esta prueba se registraron los 20 pedidos con sus respectivas cantidades de unidades. Después de cerrar el registro, las dos estaciones comenzaron a tomar pedidos del mismo pipe. La estación 1 procesó 11 pedidos y la estación 2 procesó 9, sumando entre ambas las 1100 unidades ingresadas.

Preguntas de análisis

- ¿Es posible predecir de antemano cuántos pedidos procesará cada estación?

Relaciona tu respuesta con el concepto de race condition.

No es posible saber de antemano cuántos pedidos procesará cada estación. Ambas estaciones compiten por leer los pedidos del mismo pipe y la que logra realizar la lectura primero obtiene el siguiente pedido. Esto se relaciona con una race condition, ya que el resultado depende del orden en que los procesos acceden al recurso compartido y puede cambiar en cada ejecución.

- ¿Qué pasaría si una de las estaciones no cerrará el extremo de escritura que heredó del coordinador?

El hijo podría quedarse esperando indefinidamente porque el pipe seguiría teniendo un extremo de escritura abierto. Por lo tanto, `read()` no recibiría un EOF y la estación no sabría que ya no quedan más pedidos para procesar.

Ejercicio 3

Código

Centro

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <time.h>

/*
 * Problema 3: Comunicación entre programas mediante una FIFO
 *
 * Este programa representa el centro de operaciones
 * Permanece activo durante el día esperando reportes de las sucursales
 *
 * La FIFO permite que centro.c y sucursal.c se comuniquen aunque
 * sean programas independientes y se ejecuten desde terminales
 * diferentes
 */

#define FIFO "pagos_fifo"

int main()
{
    /*
     * Creamos la FIFO si todavía no existe
     */
    if (mkfifo(FIFO, 0666) == -1) {
        printf("La FIFO ya existe, se utilizara la existente.\n");
    }

    printf("=== CENTRO DE OPERACIONES ===\n");
    printf("Esperando reportes de las sucursales...\n");

    int total_pagos = 0;

    /*
     * El centro permanece activo y vuelve a abrir la FIFO
     * cada vez que una sucursal termina de enviar su reporte
     */
    while (1) {

        /*
         * Abrimos la FIFO para lectura
         * Si no hay una sucursal conectada, el programa espera
         */
        int fd = open(FIFO, O_RDONLY);
```

```
        if (fd == -1) {
            perror("Error al abrir la FIFO");
            return 1;
        }

        char mensaje[200];
        ssize_t bytes_leidos;

        /*
         * Leemos los datos enviados por la sucursal
         */
        while ((bytes_leidos = read(fd, mensaje, sizeof(mensaje) - 1)) > 0) {

            /*
             * Agregamos el carácter final para tratar los datos
             * recibidos como una cadena de texto
             */
            mensaje[bytes_leidos] = '\0';

            /*
             * Si recibimos "cerrar", terminamos el día
             */
            if (mensaje[0] == 'c' &&
                mensaje[1] == 'e' &&
                mensaje[2] == 'r' &&
                mensaje[3] == 'r' &&
                mensaje[4] == 'a' &&
                mensaje[5] == 'r' &&
                mensaje[6] == '\0') {

                close(fd);

                printf("\n=== RESUMEN DEL DIA ===\n");
                printf("Total de pagos procesados: %d\n", total_pagos);

                unlink(FIFO);
                return 0;
            }
        }
    }
}
```

```

/*
 * El formato esperado es:
 *
 * Sucursal: nombre
 * Pagos: cantidad
 */
int pagos;
char sucursal[100];

if (sscanf(mensaje, "Sucursal: %99[^\n]\nPagos: %d",
            sucursal, &pagos) == 2) {

    time_t ahora = time(NULL);
    struct tm *hora = localtime(&ahora);

    printf("\nReporte recibido\n");
    printf("Sucursal: %s\n", sucursal);
    printf("Pagos procesados: %d\n", pagos);
    printf("Hora de llegada: %02d:%02d:%02d\n",
            hora->tm_hour,
            hora->tm_min,
            hora->tm_sec);

    total_pagos += pagos;
}
}

/*
 * La sucursal cerró su extremo de la FIFO
 * Cerramos también el nuestro y volvemos a esperar
 * por el siguiente reporte
 */
close(fd);
}

return 0;
}

```

Sucursal

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5
6 /*
7  * Problema 3: Comunicación mediante FIFO
8  *
9  * Este programa representa una sucursal
10  *
11  * La sucursal puede:
12  * 1. Enviar un reporte de pagos al centro
13  * 2. Enviar el mensaje "cerrar" para indicar que terminó el día
14  *
15  * No se utiliza fork() porque centro.c y sucursal.c son
16  * programas independientes
17  */
18
19 #define FIFO "pagos_fifo"
20
21 int main()
22 {
23     int opcion;
24
25     printf("=== SUCURSAL ===\n");
26     printf("1. Enviar reporte\n");
27     printf("2. Cerrar el día\n");
28     printf("Seleccione una opción: ");
29     scanf("%d", &opcion);
30
31     /*
32      * Primero comprobamos que la opción sea válida
33      */
34     if (opcion != 1 && opcion != 2) {
35         printf("\nOpción no válida.\n");
36         return 1;
37     }
38
39     /*
40      * Abrimos la FIFO para enviar información al centro
41      */
42     int fd = open(FIFO, O_WRONLY);
43
44     if (fd == -1) {
45         perror("Error al abrir la FIFO");
46         return 1;
47     }
48 }

```

```

    if (opcion == 1) {
        char sucursal[100];
        int pagos;

        printf("\nIngrese el nombre de la sucursal: ");
        scanf("%99s", sucursal);

        do {
            printf("Ingrese el total de pagos procesados: ");
            scanf("%d", &pagos);

            if (pagos < 0) {
                printf("La cantidad no puede ser negativa.\n");
            }

        } while (pagos < 0);

        char mensaje[200];

        /*
         * Construimos el reporte que será enviado al centro
         */
        snprintf(mensaje, sizeof(mensaje),
                 "Sucursal: %s\nPagos: %d",
                 sucursal, pagos);

        write(fd, mensaje, sizeof(mensaje));

        printf("\nReporte enviado correctamente.\n");
    }

    /*
     * Opción 2: enviar la orden de cierre
     */
    else {
        char mensaje[] = "cerrar";

        write(fd, mensaje, sizeof(mensaje));

        printf("\nSe envió la orden de cierre al centro.\n");
    }

    close(fd);

    return 0;
}

```

Ejecución

```

> ./sucursal
=== SUCURSAL ===
1. Enviar reporte
2. Cerrar el día
Seleccione una opcion: 1

Ingrese el nombre de la sucursal: Central
Ingrese el total de pagos procesados: 25

Reporte enviado correctamente.
> ./sucursal
=== SUCURSAL ===
1. Enviar reporte
2. Cerrar el día
Seleccione una opcion: 1

Ingrese el nombre de la sucursal: Zona1
Ingrese el total de pagos procesados: 40

Reporte enviado correctamente.
> ./sucursal
=== SUCURSAL ===
1. Enviar reporte
2. Cerrar el día
Seleccione una opcion: 1

Ingrese el nombre de la sucursal: Zona2
Ingrese el total de pagos procesados: 35

Reporte enviado correctamente.
/run/me/d/0/U/S/De/U 2026/8/C/S0/L/Practica 2/Practica2_S0PE52/Ejer 3 main 74

```

```
> ./centro
=== CENTRO DE OPERACIONES ===
Esperando reportes de las sucursales...

Reporte recibido
Sucursal: Central
Pagos procesados: 25
Hora de llegada: 23:39:46

Reporte recibido
Sucursal: Zonal
Pagos procesados: 40
Hora de llegada: 23:40:00

Reporte recibido
Sucursal: Zona2
Pagos procesados: 35
Hora de llegada: 23:40:07
|
```

Se comprobó que el centro puede recibir varios reportes de diferentes sucursales. En la prueba se recibieron reportes de Central con 25 pagos, Zona1 con 40 y Zona2 con 35. Además, el centro registró correctamente la hora de llegada de cada reporte y acumuló la información recibida.

Preguntas de análisis

- ¿Por qué en este caso no fue necesario usar `fork()` para relacionar `sucursal.c` con `centro.c`? Explica qué característica de las FIFO hace posible que dos programas completamente independientes se comuniquen.

No fue necesario usar `fork()` porque `centro.c` y `sucursal.c` son programas independientes que pueden ejecutarse desde terminales diferentes. Las FIFO permiten que procesos que no tienen relación entre sí se comuniquen mediante un archivo especial que funciona como una tubería con nombre.

- ¿Qué ocurre si `sucursal.c` intenta abrir la FIFO antes de que `centro.c` la haya creado?

Si la FIFO todavía no existe, `sucursal.c` no podrá abrirla y mostrará un error. En este programa, `centro.c` es el encargado de crear la FIFO mediante `mkfifo()`, por lo que primero debe ejecutarse el centro para crear el canal de comunicación.